MODULE 12 · DOMAIN 4 - TERRAFORM CONFIGURATION

Custom Conditions and Check Blocks

Four ways to assert something is true, and the one that deliberately does not stop you.

DOMAIN 4 of 8 COURSE WEIGHT 24% MODULE 12 of 27

admin@sthithapragna.com

Type constraints check the **shape** of a value. Custom conditions check whether it is **sensible**.

The four mechanisms

MECHANISM	WHEN IT RUNS, AND WHAT FAILURE DOES
Variable validation	When variable values are set. Blocks the operation
Precondition	Before the enclosing block is provisioned. Blocks
Postcondition	After the enclosing block is provisioned. Blocks
Check block	Last step of plan or apply. Warns, and continues

ON THE EXAM

Three halt the operation. The check block is the deliberate exception, and that is the distinction questions turn on.

Variable validation

- A **validation** block inside a variable block, holding a **condition** and an **error_message**.
- Available since **Terraform 0.13**. A variable may carry **several** validation blocks.
- The condition must return a **boolean**. This is where **can()** earns its place, as module nine noted.
- It is the **first line of defence**, catching a bad input before any provider is contacted.
- It **cannot see resource attributes**, because no resources exist when variables are evaluated.

> ON THE EXAM
>
> Validation runs earliest and therefore knows least. Anything requiring real infrastructure has to be a precondition instead.

Since 1.9: validation can look outside the variable

- **Before Terraform 1.9, a validation condition could reference only the variable itself.**
- **Since 1.9, conditions may reference other input variables, data sources and local values.**
- So rules like **if this flag is true, that other variable must not be empty** now live in the variable block.
- The old workaround was a **precondition on a placeholder resource**, often a null_resource.
- The exam tests **1.12**, so cross-object validation is available and the workaround is outdated.

> ON THE EXAM
>
> Older study material states flatly that validation cannot reference other variables. That was true, and it stopped being true in 1.9.

Writing the error message

- The message is what a user sees when the condition fails. It is the point of the whole exercise.
- It should be a **full sentence** describing the problem and, ideally, how to fix it.
- **Since Terraform 1.2, error messages are evaluated as expressions**, so you can interpolate values.
- That means the message can quote **what was actually supplied**, not just what was required.
- A message reading 'invalid value' wastes the mechanism entirely.

> ON THE EXAM
>
> The condition catches the mistake. The message is what determines whether the person can fix it without asking you.

Preconditions

- A **precondition** block inside **lifecycle**, holding a condition and an error message. Terraform 1.2 or later.
- Evaluated **before** the enclosing block is provisioned. If it fails, Terraform halts further evaluation.
- It documents an **assumption**: this resource only makes sense if something else is true.
- Unlike variable validation, it **can reference other resources and data sources**.
- The classic use: checking a looked-up image, subnet or certificate is suitable before building on it.

> ON THE EXAM
>
> A precondition fails before anything is created, so nothing is left half-built. That is why it is preferable to a postcondition where either would work.

Postconditions

- A **postcondition** block, also inside lifecycle. Evaluated **after** the object is provisioned or read.
- It can use **self** to refer to the enclosing object's own attributes, which a precondition cannot.
- It documents a **guarantee**: whatever this resource produced, it satisfies these properties.
- **A failure prevents changes to resources that depend on it**, so the damage stops spreading.
- Useful on a **data source** too, asserting that what was read is fit to use.

> ON THE EXAM
>
> self is the practical tell. If the condition must inspect the object itself, only a postcondition can see it.

Precondition or postcondition?

PRECONDITION

An assumption, checked first

- Runs before provisioning
- Cannot use self
- Nothing is created if it fails
- Checks inputs and other objects

POSTCONDITION

A guarantee, checked after

- Runs after provisioning or reading
- Can use self
- The object exists before it fails
- Checks the result

THE TIEBREAKER

Where both would work, prefer the precondition. Failing before creation is cheaper than failing after it.

Where each one is allowed

- **validation**: inside a variable block only.
- **precondition and postcondition**: in a resource or data source lifecycle block.
- **Output values accept a precondition** too, which is how a module checks what it is about to hand back.
- **Module blocks do not accept them.** There is no lifecycle block on a module call.
- **check**: a top-level block in the configuration, not nested in anything.

> ON THE EXAM
>
> An output precondition is the natural place for a module to assert its own contract before returning a value.

check blocks

- A top-level **check** block containing one or more **assert** blocks, each with a condition and a message.
- Introduced in **Terraform 1.5**, and executed as the **last step** of a plan or apply.
- It may contain a **scoped data source**, available only inside that check, for querying something live.
- That is how you assert about **the running system** rather than about configuration.
- The typical example: fetch a health endpoint and assert it returns success.

> ON THE EXAM
>
> A check runs after everything else, so it can see the finished result of the whole operation rather than one resource.

A check warns. It does not fail.

- **When an assertion fails, Terraform reports a warning and continues the current operation.**
- The run still succeeds. Nothing is rolled back and nothing is blocked.
- That is the point: **operational visibility without stopping a deployment**.
- So a check is the **wrong tool** for anything that genuinely must not proceed.
- HCP Terraform builds **continuous validation** on check blocks, re-running them on a schedule. Module twenty-three.

> ON THE EXAM
>
> If a scenario says the deployment must be stopped, a check block is the wrong answer no matter how well the condition fits.

What ends up in state

- **Terraform records the results of precondition and postcondition checks to state.**
- **It does not record the contents of the checks**, only whether they passed.
- So state tells you a condition held. It does not tell you what the condition was.
- Combined with module four, that is another reason to read state through supported commands rather than directly.
- Check block results surface as warnings in output, and HCP Terraform tracks them over time.

> ON THE EXAM
>
> The result is stored, the rule is not. A condition removed from configuration leaves no trace of what it once asserted.

Conditions are not policy as code

- Custom conditions live **inside the configuration they check**, written by whoever wrote it.
- So an author who does not want a rule can simply **not write it**, or delete it.
- **Sentinel and OPA policies sit outside**, are owned by a different team, and cannot be edited by the author.
- That separation is the whole point of governance. Module twenty-three covers it.
- Use conditions for **correctness**. Use policy for **enforcement across teams**.

> ON THE EXAM
>
> A condition is a module author's assertion about their own code. A policy is an organisation's rule about everybody's code.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
Reject a bad input before anything is created	Variable validation
This rule involves two different variables	Variable validation, since 1.9
Check the image is suitable before building on it	A precondition
Verify what the resource actually produced	A postcondition, using self
Alert us, but do not block the deployment	A check block
This must never be allowed to proceed	Not a check block. Use a precondition

ON THE EXAM

Two questions settle it: at what moment must this be known, and should failure stop the operation?

> A platform team wants to be told when any TLS certificate managed by their configuration is within thirty days of expiry. They run plans nightly in a pipeline.
>
> The requirement is explicit: the warning must appear in the run output, and it must never cause an unrelated deployment to fail.

Which mechanism fits?

A. A postcondition on the certificate resource

B. A check block with an assert on the expiry date

C. A variable validation comparing the expiry to the current date

D. A precondition on every resource that uses the certificate

◆ PAUSE AND DECIDE

Which mechanism fits?

A A postcondition

Would fail the operation, and worse, would block every dependent resource. An expiry warning would become a deployment outage.

B A check block

Correct. A failed assertion reports a warning and the operation continues, which is exactly the stated requirement.

C Variable validation

Runs when variables are set, before any resource exists, so it cannot see a certificate's expiry date. It would also block.

D Preconditions everywhere

Blocks, and multiplies the problem across every consumer. The certificate would take the whole configuration down with it.

THE SIGNAL

The deciding sentence is 'must never cause an unrelated deployment to fail'. Only one of the four mechanisms behaves that way.

A module takes a boolean enable_logging and a string log_bucket_name. If logging is enabled, the bucket name must not be empty. The team is on Terraform 1.12.

A senior engineer says variable validation cannot reference another variable, and proposes a null_resource carrying a precondition to enforce the rule.

How should the reviewer respond?

A Approve. The null_resource precondition is the documented pattern for cross-variable rules

B Since Terraform 1.9, validation conditions may reference other variables. Put the rule in the variable block

C Use a check block instead, since it can see both variables

D Make log_bucket_name a required variable with no default, which removes the need for a rule

◆ PAUSE AND DECIDE

How should the reviewer respond?

A Approve the workaround

It was the documented pattern, before 1.9. On 1.12 it adds a placeholder resource to state for no benefit.

B Use cross-object validation

Correct. Since 1.9, validation conditions may reference other input variables, data sources and local values.

C A check block

Warns rather than blocks, so an invalid combination would deploy anyway. It also runs last, long after the damage.

D Make it required

Forces every caller to supply a bucket name even with logging disabled, which changes the module's interface to avoid writing one rule.

THE SIGNAL

The senior engineer is not wrong, only out of date. Option A was the correct answer for several years, which is what makes it tempting.

A module looks up a machine image with a data source, then launches instances from it. The module only works with images built for the ARM architecture. Callers occasionally supply a filter that matches an x86 image, and the failure appears much later as an unhelpful provider error.

The author wants a clear message before anything is created.

Where should the condition go?

A. A validation block on the image filter variable

B. A precondition on the instance resource, checking the data source's architecture attribute

C. A postcondition on the instance resource using self

D. A check block asserting the architecture after apply

◆ PAUSE AND DECIDE

Where should the condition go?

A Validation on the filter

Validation runs before anything is read, so it can inspect the filter string but never the image the filter matched. The filter is not the problem.

B Precondition on the instance

Correct. By then the data source has been read, so the architecture is knowable, and failing here creates nothing.

C Postcondition with self

Runs after the instance is provisioned. The stated requirement is a clear message before anything is created.

D A check block

Runs last and only warns, so the wrong instances would be built and the operation would succeed.

THE SIGNAL

A postcondition on the data source would also work, and is arguably better: it fails at the point the wrong image was read.

> A security team requires that no storage bucket created by a shared module can ever be publicly readable. They add a check block with an assert on the bucket's access setting, and a clearly worded error message.
>
> Three weeks later, a bucket is deployed with public access. The pipeline run that created it is marked successful, and the warning is in the log.

What went wrong, and what should replace it?

A. The assert condition was written incorrectly and evaluated true

B. A check block only warns. Use a precondition or postcondition, which block the operation

C. Check blocks only run during plan, not apply, so the apply was unaffected

D. The check block needed a scoped data source to read the live bucket setting

◆ PAUSE AND DECIDE

What went wrong, and what should replace it?

A Condition was wrong

The warning is in the log, which means the assertion evaluated false and fired correctly. The mechanism worked; the choice of mechanism did not.

B Checks warn, conditions block

Correct. A failed assertion reports a warning and the operation continues. Enforcement requires a mechanism that halts.

C Only run during plan

False. A check block executes as the last step of plan and of apply. Both were unaffected because warnings never block.

D Needed a scoped data source

Would change what the check can see, not what a failure does. A better-informed warning is still a warning.

THE SIGNAL

The deliberate inverse of scenario 1. Same mechanism, opposite requirement, opposite answer. Read what failure must do.

Module 12 recap

- **Four mechanisms, four moments.** Validation, precondition, postcondition, check.
- **Only the check block warns.** The other three halt the operation.
- **Since 1.9, validation can reference other variables, data sources and locals.**
- **Only a postcondition has self.** If the condition inspects the object itself, that is the answer.
- **Prefer the earliest mechanism that can see what it needs to see.**
- **Module blocks accept none of them.** Outputs accept a precondition.

> ON THE EXAM
>
> State records that a condition passed, not what it asserted. The rule lives only in your configuration.